

Wordle 2.1 - DSA Topics and File Documentation

Table of Contents

1. [DSA Topics Used](#)
 2. [File Documentation](#)
 3. [Project Structure Overview](#)
-

DSA Topics Used

1. Arrays (Static Arrays)

- **Where Used:**

- `Wordle.cpp`: `dictionary[MAX_WORDS][MAX_WORD_LENGTH]` - 2D character array storing word lists
- `Wordle_wordlists.cpp`: Three 2D character arrays (`easyWords`, `mediumWords`, `hardWords`) storing word lists
- `Wordle.cpp`: `secretWord[MAX_WORD_LENGTH]` - Character array for the secret word
- `Wordle.cpp`: `guess[MAX_WORD_LENGTH]` - Character array for user guesses
- `Wordle.cpp`: Boolean arrays `usedSecret[]` and `usedGuess[]` for tracking letter usage

- **Why Used:**

- Arrays provide fast, direct access to word data
- Fixed-size arrays are memory-efficient for known word counts
- 2D arrays naturally represent collections of strings (words)
- Character arrays (C-style strings) are used for compatibility with C string functions

2. Doubly Linked List

- **Where Used:**

- `GuessList.h` and `GuessList.cpp`: `DoublyNode` structure with `next` and `prev` pointers
- `GuessList` class maintains a doubly linked list of all user guesses

- **Why Used:**

- Allows efficient insertion of new guesses at the end ($O(1)$)
- Enables bidirectional traversal (forward and backward)
- Supports `displayReverse()` method to show guesses in reverse order
- Demonstrates DSA concepts while maintaining guess history

3. Circular Linked List

- **Where Used:**

- `GuessList.cpp`: The doubly linked list is made circular by connecting `tail->next` to `head` and `head->prev` to `tail`
- `displayCircular()` method demonstrates circular traversal

- **Why Used:**

- Demonstrates circular data structure concepts
- Allows continuous traversal through the guess history
- Educational purpose to showcase DSA implementation

4. Stack (LIFO - Last In First Out)

- **Where Used:**

- `Wordle.h`: `std::stack<GuessRecord> guessStack` - STL stack container
- `Wordle.cpp`: Pushes each guess to the stack in `play()` method

- **Why Used:**

- Stores guess history in reverse chronological order (most recent first)
- Could be used for undo functionality (though not currently implemented)
- Demonstrates stack data structure usage
- Natural fit for tracking sequential game actions

5. Queue (FIFO - First In First Out)

- **Where Used:**

- `Wordle.h`: `std::queue<HintNode> hintQueue` - STL queue container
- `Wordle.cpp`: `initializeHintQueue()` and `giveHint()` methods

- **Why Used:**

- Manages hint system by storing letter positions in order
- Ensures hints are revealed in a predictable sequence (FIFO)
- First hint is free, subsequent hints cost a life
- Queue naturally handles sequential hint distribution

6. Strings (C-style and C++ strings)

- **Where Used:**

- C-style strings: Throughout the project for word storage (`char[]` arrays)
- C++ strings: `std::string` used in `main.cpp` and `Wordle.cpp` for input handling
- String operations: `strcpy_s()`, `strcmp()`, `input.length()`, etc.

- **Why Used:**

- C-style strings for compatibility with existing word list arrays
- C++ strings for safer and more convenient input handling
- String comparison for checking guesses against secret word

7. File I/O Operations

- **Where Used:**

- `Wordle.cpp`: `getBestScore()` and `saveBestScore()` methods
- Files: `bestscore_easy.txt`, `bestscore_medium.txt`, `bestscore_hard.txt`

- **Why Used:**

- Persist best scores across game sessions
- Separate files for each difficulty level
- Simple file-based data persistence

8. Random Number Generation Algorithms

- **Where Used:**

- `Wordle.cpp`: `std::mt19937` (Mersenne Twister) for selecting random secret word
- `Wordle.cpp`: `rand()` for randomizing hint positions

- **Why Used:**

- Ensures different secret word each game
- Random hint order prevents predictable hint patterns
- Modern RNG (`mt19937`) provides better randomness than `rand()`

9. Enumeration (Enum)

- **Where Used:**

- `Wordle.h`: `enum Difficulty { EASY = 1, MEDIUM = 2, HARD = 3 }`

- **Why Used:**

- Type-safe way to represent difficulty levels
- Better than using magic numbers (1, 2, 3)
- Improves code readability and maintainability

File Documentation

main.cpp

Purpose: Entry point of the program and main menu system

What it does:

- Displays the main menu with options to play, view instructions, or exit
- Handles user input for menu choices
- Manages the game loop and difficulty selection
- Displays "How to Play" instructions
- Handles exception catching for error management

- Prompts user to play again after each game

Why it exists:

- Separates user interface logic from game logic
- Provides a clean entry point for the application
- Manages the overall program flow and user interaction
- Acts as the controller between user and game engine

Key Functions:

- `main()`: Program entry point, menu loop
 - `howToPlay()`: Displays game instructions
 - `printBorder()`: Utility for visual formatting
-

Wordle.h

Purpose: Header file defining the main `Wordle` game class and related structures

What it contains:

- `Difficulty` enum: Defines game difficulty levels (EASY, MEDIUM, HARD)
- `GuessRecord` struct: Stores guess data for stack operations
- `HintNode` struct: Stores hint information (position and letter) for queue operations
- `Wordle` class declaration: Main game class with all game logic

Why it exists:

- Provides interface/declaration for the Wordle game class
- Defines data structures used throughout the game
- Separates interface from implementation (header/implementation pattern)
- Allows other files to include and use the Wordle class

Key Components:

- Private members: Dictionary arrays, secret word, game state variables, DSA structures (stack, queue, linked list)
 - Public methods: Game control (`resetGame()`, `play()`), hint system (`giveHint()`), score management (`getBestScore()`, `saveBestScore()`)
-

Wordle.cpp

Purpose: Implementation of the main Wordle game logic

What it does:

- Implements all `Wordle` class methods
- Manages game state (difficulty, attempts, word selection)
- Handles word selection from difficulty-specific word lists
- Implements guess checking algorithm with color coding (green/yellow/gray)

- Manages hint system using queue data structure
- Tracks guesses using stack and linked list
- Handles file I/O for best score persistence
- Displays game UI with colored output

Why it exists:

- Contains the core game logic implementation
- Separates game mechanics from user interface
- Implements DSA structures (stack, queue) for game features
- Handles all game state management

Key Functions:

- `resetGame()`: Initializes game with selected difficulty, loads word list, selects secret word
- `play()`: Main game loop, handles user input, manages attempts
- `checkGuess()`: Compares guess to secret word, determines letter colors (green/yellow/gray)
- `giveHint()`: Uses queue to provide hints, first free, rest cost a life
- `initializeHintQueue()`: Populates queue with random letter positions from secret word
- `getBestScore()` / `saveBestScore()`: File I/O for persistent score tracking

DSA Usage:

- **Stack**: Stores guess history (`guessStack`)
 - **Queue**: Manages hint system (`hintQueue`)
 - **Arrays**: Word storage (`dictionary[ ][ ]`), character arrays for words
 - **Linked List**: Uses `GuessList` object to track guesses
-

GuessList.h

Purpose: Header file for the guess history data structure

What it contains:

- `DoublyNode` struct: Node for doubly linked list with `next` and `prev` pointers
- `GuessList` class declaration: Doubly linked list with circular reference

Why it exists:

- Defines the data structure for maintaining guess history
- Demonstrates doubly linked list and circular linked list concepts
- Provides interface for guess tracking functionality
- Separates data structure implementation from game logic

Key Components:

- `DoublyNode`: Stores guess string and pointers to next/previous nodes
 - `GuessList`: Maintains head, tail, and size; provides methods for adding, displaying, and clearing guesses
-

GuessList.cpp

Purpose: Implementation of the doubly linked list with circular reference for guess history

What it does:

- Implements doubly linked list operations (insertion, traversal)
- Maintains circular reference (tail points to head, head's prev points to tail)
- Provides forward traversal (`displayGuesses()`)
- Provides reverse traversal (`displayReverse()`) - demonstrates doubly linked list advantage
- Provides circular traversal (`displayCircular()`) - demonstrates circular linked list
- Manages memory (constructor, destructor, clear method)

Why it exists:

- Demonstrates DSA concepts (doubly linked list, circular linked list)
- Maintains history of all user guesses
- Provides flexible traversal options (forward, backward, circular)
- Separates data structure implementation from game logic

Key Functions:

- `addGuess()`: Adds new guess to the end, maintains circular structure
- `displayGuesses()`: Forward traversal showing all guesses
- `displayReverse()`: Backward traversal (demonstrates doubly linked list)
- `displayCircular()`: Circular traversal for multiple rounds (demonstrates circular linked list)
- `clear()`: Frees all memory, breaks circular link before deletion
- `~GuessList()`: Destructor ensures proper cleanup

DSA Usage:

- **Doubly Linked List:** Each node has both `next` and `prev` pointers
 - **Circular Linked List:** Last node points to first, first node's prev points to last
 - **Dynamic Memory:** Uses `new/delete` for node allocation
-

Wordle_wordlists.h

Purpose: Header file declaring external word list arrays

What it contains:

- External declarations for word count constants (`EASY_WORD_COUNT`, `MEDIUM_WORD_COUNT`, `HARD_WORD_COUNT`)
- External declarations for word arrays (`easyWords`, `mediumWords`, `hardWords`)

Why it exists:

- Separates word list declarations from implementation
- Allows other files to access word lists without including the large array definitions
- Follows C++ best practices (declaration in header, definition in source)
- Reduces compilation time (large arrays only compiled once in .cpp file)

Key Components:

- Three word arrays for three difficulty levels
 - Word count constants for each difficulty
-

Wordle_wordlists.cpp

Purpose: Contains the actual word list data (large arrays of words)

What it contains:

- `easyWords[ ][ ]`: 839 four-letter words
- `mediumWords[ ][ ]`: 2,309 five-letter words
- `hardWords[ ][ ]`: 846 six-letter words
- Word count constants calculated from array sizes

Why it exists:

- Stores all valid words for the game
- Separates large data from code logic
- Reduces compilation time (large arrays compiled separately)
- Makes it easy to update word lists without modifying game logic
- Auto-generated from word source files

Key Features:

- Static arrays with fixed sizes
 - Words organized by difficulty (length)
 - Word counts automatically calculated using `sizeof()` operator
-

colors.h

Purpose: Defines ANSI color codes for terminal output

What it contains:

- Color macro definitions: `GREEN`, `YELLOW`, `GRAY`, `RED`, `BLUE`, `RESET`
- ANSI escape sequences for colored terminal output

Why it exists:

- Centralizes color definitions for consistent use
- Makes code more readable (using `GREEN` instead of escape sequences)
- Easy to modify colors in one place
- Provides visual feedback in the game (green = correct, yellow = wrong position, gray = not in word)

Usage:

- Used throughout the project for colored terminal output
- Enhances user experience with visual feedback

- Makes game output more engaging and easier to read
-

config.h

Purpose: Central configuration file with game constants

What it contains:

- `MAX_WORDS`: Maximum number of words that can be stored (1000)
- `MAX_WORD_LENGTH`: Maximum word length including null terminator (7)
- Word length constants: `EASY_WORD_LEN` (4), `MEDIUM_WORD_LEN` (5), `HARD_WORD_LEN` (6)
- Attempt constants: `EASY_ATTEMPTS` (5), `MEDIUM_ATTEMPTS` (6), `HARD_ATTEMPTS` (7)

Why it exists:

- Centralizes all configuration constants
- Makes it easy to modify game parameters in one place
- Prevents magic numbers scattered throughout code
- Improves maintainability and readability
- Ensures consistency across all files

Key Benefits:

- Single source of truth for game configuration
 - Easy to adjust difficulty parameters
 - Reduces errors from inconsistent values
 - Makes code self-documenting
-

Project Structure Overview

Architecture Pattern:

The project follows a modular design with clear separation of concerns:

1. **Entry Point:** `main.cpp` - User interface and program flow
2. **Game Logic:** `Wordle.h/cpp` - Core game mechanics
3. **Data Structures:** `GuessList.h/cpp` - Guess history management
4. **Data:** `Wordle_wordlists.h/cpp` - Word list storage
5. **Configuration:** `config.h` - Game constants
6. **Utilities:** `colors.h` - Terminal formatting

Data Flow:

1. User selects difficulty in `main.cpp`
2. `Wordle::resetGame()` loads appropriate word list from `Wordle_wordlists.cpp`
3. Secret word selected randomly
4. User makes guesses in `Wordle::play()`
5. Each guess added to `GuessList` (doubly linked list) and `guessStack` (stack)
6. `Wordle::checkGuess()` compares guess to secret word

7. Hints provided via `hintQueue` (queue)
8. Best scores saved/loaded from files

DSA Integration Summary:

- **Arrays:** Word storage and game state
- **Doubly Linked List:** Guess history with bidirectional traversal
- **Circular Linked List:** Continuous traversal capability
- **Stack:** Guess history for potential undo feature
- **Queue:** Sequential hint distribution
- **Strings:** Word representation and manipulation
- **File I/O:** Persistent data storage

This project effectively demonstrates multiple DSA concepts in a practical, real-world application while maintaining clean code organization and separation of concerns.